

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 7

Three Small Programs: Processes, Redirection, a Pipe

Ciur Alexandrina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 Purpose and Theory **2**
1.1 Purpose 2
1.2 Short theory 2

2 Work Done **2**
2.1 Part 1 – fork, exec and wait (`run.c`) 2
2.2 Part 2 – A zombie on purpose (`zombie.c`) 3
2.3 Part 3 – Redirection (`redirect.c`) 4
2.4 Part 4 – A pipe (`pipeline.c`) 4

3 Conclusions **5**

1 Purpose and Theory

1.1 Purpose

The lab answers three questions from the lecture by running real code on Linux: how a new process is started, where a program's output actually goes, and how the shell connects two commands with `|`. Four short C programs reproduce, step by step, what the shell does for `ls -l`, `ls -l > out.txt` and `ls /etc | wc -l`.

1.2 Short theory

`fork()` duplicates the calling process. It returns `0` in the child and the child's PID in the parent.

`exec*()` loads a new program into the current process. The PID and the table of open descriptors stay the same; the old code is gone, so on success `exec` never returns.

`waitpid()` lets the parent collect the exit status of a child. Until that happens, a child that has already ended stays in the process table as a *zombie*.

File descriptors are small integers: `0` = stdin, `1` = stdout, `2` = stderr. `open()` and `dup()` always return the *smallest free* number, which is the whole trick behind redirection.

`pipe(p)` creates a kernel buffer with a read end `p[0]` and a write end `p[1]`. The reader gets end-of-file only after *every* copy of `p[1]` is closed.

2 Work Done

2.1 Part 1 – fork, exec and wait (`run.c`)

The parent prints its PID, forks, and the child turns itself into `ls -l`. The parent waits and prints the child's exit status.

Listing 1: run.c with `ls -l`

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 14503
child:  my PID is 14504, my parent is 14503
total 344
-rw-rw-r-- 1 vboxuser vboxuser   2194 Oct 25 18:33  lab7_run.sh
-rw-rw-r-- 1 vboxuser vboxuser 249768 Oct 25 17:14 'lab_os (7).pdf'
-rw-r--r-- 1 vboxuser vboxuser    675 Oct 25 18:35  out.txt
-rw-rw-r-- 1 vboxuser vboxuser   2362 Oct 25 18:35  outputs.txt
-rwxrwxr-x 1 vboxuser vboxuser  16296 Oct 25 18:35  pipeline
-rw-rw-r-- 1 vboxuser vboxuser   1022 Oct 25 17:14  pipeline.c
-rwxrwxr-x 1 vboxuser vboxuser  16336 Oct 25 18:35  redirect
-rw-rw-r-- 1 vboxuser vboxuser    720 Oct 25 17:14  redirect.c
-rwxrwxr-x 1 vboxuser vboxuser  16392 Oct 25 18:45  run
-rw-rw-r-- 1 vboxuser vboxuser    601 Oct 25 18:44  run.c
-rwxrwxr-x 1 vboxuser vboxuser  16432 Oct 25 18:35  zombie
-rw-rw-r-- 1 vboxuser vboxuser    745 Oct 25 17:14  zombie.c
parent: child 14504 ended with status 0
```

Then the `execlp` line was changed twice:

```
execlp("date", "date", (char *)NULL);
execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

Listing 2: Variants with `date` and `nosuchprog`

```
$ ./run          # date
parent: my PID is 14572
child:  my PID is 14573, my parent is 14572
Sun Oct 25 06:46:09 PM UTC 2026
parent: child 14573 ended with status 0

$ ./run          # nosuchprog
parent: my PID is 14592
child:  my PID is 14593, my parent is 14592
exec: No such file or directory
parent: child 14593 ended with status 127
```

Observations – Part 1

1. When `execlp` succeeds, the child's code is replaced by `ls` (or `date`), so the instruction after it simply no longer exists and is never reached. With `nosuchprog` the program is not found in `PATH`, `execlp` returns -1 with `errno = ENOENT`, and only then `perror("exec")` prints *No such file or directory*.

2. The child prints the value of `getppid()`. It is the same number the parent printed on the first line, i.e. the PID of the `./run` process that called `fork()` (e.g. child 14504 reports parent 14503).

3. With `ls` (and `date`) the status is `0`. With `nosuchprog` it is `127`, which comes from our own `exit(127)` – the same code bash uses for "command not found".

2.2 Part 2 – A zombie on purpose (`zombie.c`)

`zombie.c` is `run.c` with one extra line, `sleep(20);`, placed right before `waitpid`. The child (`ls`) finishes almost at once, but nobody collects it for 20 seconds.

Listing 3: Process table during and after the sleep

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   5346    5259 Ss   /usr/bin/bash
  12767    5346 S+   bash lab7_run.sh
  12768    5346 S+   tee outputs.txt
  12797   12767 S+   ./zombie
  12799   12797 Z+   [ls] <defunct>
  12801   12767 R+   ps -o pid,ppid,stat,cmd

$ ps -o pid,ppid,stat,cmd        # after 20 seconds
    PID    PPID STAT CMD
   5346    5259 Ss   /usr/bin/bash
  12767    5346 S+   bash lab7_run.sh
  12768    5346 S+   tee outputs.txt
  12808   12767 R+   ps -o pid,ppid,stat,cmd
```

Observations – Part 2

1. `ls` (PID 12799, child of `./zombie` 12797) has the letter **Z** in `STAT` (shown as `Z+`; the `+` only means it is in the foreground process group): a zombie (`<defunct>`). The process has ended and its memory is freed, but the kernel keeps its entry (PID and exit status) until the parent asks for it.

2. After 20 seconds the `[ls] <defunct>` line is gone. It was removed by `waitpid(pid, &status, 0);` – the parent read the exit status and the kernel freed the entry.

2.3 Part 3 – Redirection (`redirect.c`)

The child closes descriptor 1, opens `out.txt` and runs `ls -l`.

Listing 4: Running redirect and checking out.txt

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 324
-rw-rw-r-- 1 vboxuser vboxuser   2194 Oct 25 18:33 lab7_run.sh
-rw-rw-r-- 1 vboxuser vboxuser 249768 Oct 25 17:14 lab_os (7).pdf
```

Observations – Part 3

1. `open` returned **1**. Descriptors 0 and 2 were still in use, 1 had just been closed, and `open` always takes the smallest free number.

2. `ls` writes to descriptor 1 as always; it does not know what that descriptor points to. `exec` keeps the descriptor table, so in the child 1 now means `out.txt`, not the terminal. The two lines that did appear on screen come from `stderr` (2) and from the parent, whose descriptor 1 was never touched.

3. For `2> err.txt` we would close descriptor **2** before `open`; the file would then receive number 2.

2.4 Part 4 – A pipe (`pipeline.c`)

One pipe and two children: in the first, descriptor 1 becomes `p[1]` and it runs `ls /etc`; in the second, descriptor 0 becomes `p[0]` and it runs `wc -l`. `dup` works like `open`: it picks the smallest free number.

Listing 5: Hand-made pipeline vs. the shell

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
236
$ ls /etc | wc -l
236
```

Then, in the parent, the line `close(p[1]);` was commented out:

```
close(p[0]);      /* the parent uses neither end */
/* close(p[1]); */
```

Listing 6: The broken version never finishes

```
$ ./pipeline   # with close(p[1]) commented out in the parent
$ ps -o pid,ppid,stat,cmd   # the program does not end
    PID    PPID STAT CMD
   5346    5259 Ss   /usr/bin/bash
  12767    5346 S+   bash lab7_run.sh
  12768    5346 S+   tee outputs.txt
  12833   12767 S+   /tmp/pipeline_broken
  12836   12833 S+   wc -l
  12837   12767 R+   ps -o pid,ppid,stat,cmd
$ kill 12833          # stop the parent (same effect as Ctrl-C)
236
```

Observations – Part 4

1. Yes, `./pipeline` and `ls /etc | wc -l` print the same number (236).
2. `wc` never ends; `ls` finishes normally and is no longer in the list.
3. `wc` waits for end-of-file on the pipe, and EOF never comes because the parent still keeps the write end `p[1]` open (while the parent itself is stuck in `wait` for `wc`). The run confirms it: as soon as the parent was killed, the kernel closed its copy of `p[1]`, `wc` received EOF and immediately printed `236`.

3 Conclusions

The four programs showed that the shell does not have any special power: everything it does for a command line is built from a handful of system calls. `fork` creates the process, `exec` gives it a new program, and `waitpid` both returns the exit status and removes the zombie from the process table – skipping it, even for a few seconds, leaves a `Z` entry behind.

Redirection and pipes turned out to rely on the same simple rule: a new descriptor always gets the smallest free number. Closing 1 (or 0) right before `open` or `dup` is enough to send a program's output to a file or into a pipe, without changing the program at all. The broken pipeline was the most useful part: it showed that every unused pipe end must be closed, otherwise the reader never sees end-of-file and the whole chain blocks.